

GESTION DE CATÁLOGO DE PRODUCTOS DE SUPERMERCADO

ESTRUCTURAS DE DATOS



REPORTE TÉCNICO

Proyecto: Sistema de Gestión de Catálogo de Productos de Supermercado

I. DISEÑO DE LAS ESTRUCTURAS UTILIZADAS

El núcleo (backend) del sistema fue diseñado bajo una arquitectura de índices múltiples en memoria dinámica, complementada con estructuras de enrutamiento espacial.

- **Tabla Hash (Código de Barras):** Implementada mediante arreglos dinámicos y resolución de colisiones por encadenamiento (Separate Chaining). Actúa como la Llave Primaria (Primary Key) del ecosistema de inventario.
- **Árbol AVL (Nombres):** Estructura binaria auto-balanceable que mantiene el catálogo estrictamente ordenado alfabéticamente en todo momento, controlando el factor de equilibrio mediante rotaciones simples y dobles.
- **Árbol B (Fechas de Expiración):** Árbol multidimensional configurado con un grado mínimo $t=2$. Agrupa múltiples claves por nodo para optimizar las búsquedas en rangos de fechas específicos.
- **Árbol B+ (Categorías):** Variante del Árbol B (también con $t=2$) que confina los objetos de inventario exclusivamente en sus nodos hoja, los cuales están enlazados secuencialmente para permitir extracciones masivas e iterativas.
- **Grafos (Red Logística Nacional):** Topología no lineal que modela el país. Los vértices representan las sucursales físicas y las aristas representan las conexiones logísticas (carreteras), almacenando pesos bidimensionales: tiempo de tránsito y costo de flete.
- **Colas / Queues (Simulador de Despacho):** Estructuras lineales de tipo FIFO (First-In, First-Out). Modelan el embudo de procesamiento interno de las sucursales físicas, divididas en colas de Recepción, Preparación y Salida.
- **Pilas / Stacks (Historial Transaccional):** Estructura lineal estricta LIFO (Last-In, First-Out) que actúa como memoria temporal de la sesión para el sistema de "Rollback" (deshacer operaciones).

2. JUSTIFICACIÓN DE LAS ESTRUCTURAS IMPLEMENTADAS

La decisión arquitectónica de mantener un único objeto residente en el *Heap* y ser referenciado simultáneamente por estas estructuras responde al principio de "**trade-off**" estructural:

- **Por qué Hash y no Búsqueda Lineal:** En operaciones de punto de venta logístico, la latencia debe ser nula. El Hash ofrece acceso determinista. Se descartó el direccionamiento abierto para evitar el efecto de *Aglomeración Primaria* y el colapso por marcas de borrado (*Tombstones*).
- **Por qué Árboles (AVL, B, B+):** Mientras el Hash es ciego ante rangos y ordenamientos, los árboles estructuran la semántica de los datos. El AVL garantiza reportes alfabéticos inmediatos. El Árbol B+ fue vital porque, de forma análoga a una consulta SQL indexada, permite encontrar la primera coincidencia de una categoría y simplemente "barrer" la lista enlazada inferior sin volver a recorrer el árbol superior.
- **Por qué Grafos y Algoritmo de Dijkstra:** Emulando el comportamiento del enrutamiento de paquetes en telecomunicaciones (OSPF o protocolos link-state), el Grafo permite iterar sobre las rutas más eficientes. Dijkstra fue elegido para garantizar el camino más corto evaluando dinámicamente si el usuario prioriza el *Tiempo* (urgencia) o el *Costo* (economía).

- **Por qué Colas y Pilas:** El mundo físico no es concurrente perfecto. Las Colas absorben los cuellos de botella de la vida real (Tiempos de Ingreso y Despacho), mientras que la Pila garantiza la integridad del sistema permitiendo revertir el estado exacto del último ingreso sin afectar el resto del índice.

3. ANÁLISIS DE COMPLEJIDAD (NOTACIÓN BIG-O)

La siguiente tabla resume las garantías matemáticas del comportamiento de las estructuras ante la escalabilidad (n = número de productos, V = sucursales, E = conexiones, k = elementos en rango):

Estructura	Inserción	Eliminación	Búsqueda (Peor)	Búsqueda (Promedio)	Justificación
Tabla Hash	$O(1)$	$O(1)^*$	$O(n)$	$O(1)$	Indexación mediante función matemática directa. Peor caso asumido solo ante colisiones masivas en la misma ranura.
Árbol AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	La profundidad estructural está matemáticamente acotada; descarta la mitad del espectro en cada nivel.
Árbol B	$O(\log_t n)$	$O(\log_t n)$	$O(\log_t n)$	$O(\log_t n)$	Crecimiento horizontal que reduce la altura general, requiriendo búsqueda lineal interna en nodos de tamaño constante.
Árbol B+	$O(\log_t n)$	$O(\log_t n)$	$O(k)$	$O(\log_t n)$	La localización inicial es logarítmica; la extracción de la categoría completa toma $O(k)$, siendo k el tamaño del subconjunto.
Lista Simple	$O(1)$	$O(n)$	$O(n)$	$O(n)$	Inserción directa en cabeza, pero condena al procesador a iterar ciegamente para buscar o eliminar.
Grafos (Dijkstra)	$O(1)$	$O(1)$	$O(E + V \log V)$	$O(V^2)$	La exploración de caminos depende de la densidad de conexiones.
Cola / Pila	$O(1)$	$O(1)$			Acceso restringido exclusivamente a los extremos (Head/Tail o Top).

4. METODOLOGÍA DE MEDICIÓN DE RENDIMIENTO

Para aislar el desempeño teórico del empírico y mitigar el ruido del planificador de tareas del Sistema Operativo, el motor de medición utiliza la siguiente metodología:

1. **Librería de Precisión:** Uso de `std::chrono::high_resolution_clock` de C++ para capturar marcas de tiempo de CPU.
2. **Conversión Escalar:** Los deltas de tiempo se capturan nativamente en *nanosegundos* y se convierten aritméticamente a *microsegundos* (μs) (división flotante / 1000.0) para lograr precisión de hasta 3 decimales, evitando los falsos 0 μs en operaciones $O(1)$.
3. **Tamaño de Muestra:** Benchmarks ejecutados sobre $N = 1000$ cargas iniciales y $M = 5$ repeticiones por consulta analítica.

5. RESULTADOS EXPERIMENTALES

5.1. Tiempos Promedio de Inserción y Eliminación

Parámetros de prueba: Carga masiva de 1,000 registros.

Estructura	Inserción (μs)	Eliminación (μs)	Comportamiento Observado
Tabla Hash	0.42 μs	2.10 μs	Mantiene latencia casi imperceptible.
Lista No Ordenada	0.10 μs	100.00 μs	Espejismo lineal: Inserción inmediata, eliminación catastrófica. Al insertar siempre en la cabeza (<code>insertFront</code>), no hace comparaciones, solo asigna memoria. Es ideal para logs rápidos, pero inútil como índice.
Árbol AVL	1.04 μs	5.80 μs	La inversión de tiempo en rotaciones asegura una eliminación rápida.
Árbol B	1.69 μs	2.00 μs	Representa el costo real de mantener un índice robusto. Dedicamos microsegundos valiosos a dividir nodos (<code>splits</code>) y ejecutar rotaciones y balanceo.
Árbol B+	0.29 μs	10 μs	Esto se justifica teóricamente porque, aunque hay 1,000 productos, existen muy pocas categorías únicas. El árbol no crece verticalmente; simplemente encuentra la hoja existente en tiempo mínimo y añade el producto a la lista enlazada inferior en $O(1)$.

5.2. Escenarios de Búsqueda de Estrés

Comparativa entre casos de éxito (aleatorio) y el peor caso absoluto (inexistencia).

Escenario	Hash $O(1)$	AVL $O(\log n)$	Lista $O(n)$
Búsqueda Exitosa Aleatoria	1.000 μs	0.412 μs	1.710 μs
Búsqueda Fallida (No Existe)	0.026 μs	0.477 μs	11.723 μs
Extremo Superior (Peor Caso)	0.485 μs	0.740 μs	12.480 μs

5.3. Latencia de Transferencias (Simulación Logística)

Al integrar la búsqueda, eliminación en origen y el algoritmo de grafos, las operaciones logísticas arrojaron:

- **Tiempo de Enrutamiento (Dijkstra):** Dinámico dependiendo de los saltos.
 - **Penalización FIFO:** Las transferencias suman directamente el t_{ingreso} , t_{traspaso} y t_{despacho} de cada nodo recorrido, emulando la latencia del mundo físico.
-

6. COMPARACIÓN ANALÍTICA ENTRE ESTRUCTURAS

1. **Indexación Constante vs Lineal:** La Lista demostró que carecer de un índice es insostenible para un sistema de grado comercial. Aunque insertar un dato toma solo $0.10 \mu s$, eliminarlo penalizó a la CPU con casi $100 \mu s$ al verse forzada a inspeccionar toda la memoria secuencialmente. El Hash dominó este aspecto eliminando el dato en tan solo $2.1 \mu s$.
 2. **Consistencia Logarítmica:** El Árbol AVL es la prueba empírica de la estabilidad. Sin importar si se buscó el primer producto alfabético, el último, o uno inexistente, la latencia jamás superó el umbral de $0.80 \mu s$.
 3. **Memoria vs CPU:** Mantener Árboles B/B+ y AVL requiere llamadas constantes para la reserva dinámica de nuevos nodos internos y rotaciones, provocando tiempos de inserción marginalmente más altos ($1.04 \mu s$ a $1.69 \mu s$). No obstante, es un pago por adelantado (trade-off) que protege al sistema de cuellos de botella severos en tiempo de ejecución de consultas pesadas.
-

7. CONCLUSIONES

1. **Supremacía No Lineal:** El ecosistema del sistema comprueba que las estructuras no lineales son de implementación obligatoria para el escalamiento transaccional. La Tabla Hash funciona perfectamente como pilar del CRUD directo, mientras que los Árboles suplen las carencias del Hash respecto al ordenamiento e iteraciones jerárquicas.
2. **Sinergia de Datos y Redes:** La integración de árboles para la memoria local y Grafos ponderados para la interconexión nacional genera un sistema logístico sumamente robusto. Dijkstra garantiza el enrutamiento matemático perfecto entre sucursales, operando de manera análoga al cálculo del camino más corto en las redes de telecomunicación.
3. **Fiabilidad Diagnóstica:** El módulo de Gestion y Rendimiento a escala de nanosegundos valida científicamente que el diseño del backend puede absorber altos picos de estrés transaccional, garantizando latencias ultra-bajas independientemente de las condiciones del volumen de datos a procesar.